

Universal Verification Methodology in System Verilog

José Eduardo Chim Cano

Embedded Systems 8^oA

UPY

Mérida, Yucatán

2309057@upy.edu.mx

Abstract—This work presents the design, verification, and FPGA implementation of a parameterizable Arithmetic Logic Unit (ALU) in SystemVerilog. A modular verification environment based on a virtual interface was developed to decouple the testbench from the Device Under Test (DUT), following principles adapted from the Universal Verification Methodology (UVM). The ALU was first verified through simulation, confirming correct arithmetic results and status flag behavior, and was subsequently synthesized and implemented on an Altera/Intel DE2-115 development board (Cyclone IV E FPGA) using Quartus Prime.

Index Terms—SystemVerilog, ALU, virtual interface, UVM, FPGA, DE2-115, RTL verification

I. INTRODUCTION

Verifying digital designs through directly connected testbenches becomes impractical as design complexity grows, since every change to the Device Under Test (DUT) requires reworking the entire testbench. Modern verification methodologies, such as the Universal Verification Methodology (UVM), address this problem through modularity: reusable components communicate with the DUT exclusively through a virtual interface, allowing the testbench infrastructure to remain unchanged even when the DUT itself is modified. This project applies these principles at a reduced scale to verify a parameterizable Arithmetic Logic Unit (ALU), and subsequently implements the verified design on a DE2-115 FPGA development board.

II. THEORETICAL FRAMEWORK

An Arithmetic Logic Unit (ALU) is a combinational-sequential circuit capable of performing arithmetic and logical operations selected by a control code.

The ALU implemented in this project uses two operands x_i and y_i of parameterizable width (`DataWidth`), six single-bit control signals (zx_i , nx_i , zy_i , ny_i , f_i , no_i) that together form a 6-bit instruction code, and produces an output result (`out_o`) along with three status flags: `zr_o` (zero), `ng_o` (negative), and `pos_o` (positive).

The operations implemented are shown in Table I.

TABLE I
ALU OPERATIONS

Control code (zx nx zy ny f no)	Operation
101010	Zero
111111	One
111010	Minus one
001100	x_i
110000	y_i
001101	NOT x_i
110001	NOT y_i
001111	$-x_i$
110011	$-y_i$
000010	$x_i + y_i$
010011	$x_i - y_i$
000111	$y_i - x_i$
000000	x_i AND y_i
010101	x_i OR y_i

Unlike a conventional single-opcode ALU, this design selects the operation through six independent control bits that manipulate the operands before combining them, following a structure comparable to the ALU described in the Nand2Tetris/HACK architecture.

To verify this design, a virtual interface was used as the communication bridge between the hardware domain (RTL, described in SystemVerilog) and the software domain (the object-oriented testbench). The virtual interface groups all DUT signals into a single reusable block and defines a clocking block, which applies a small input/output skew (1 ns) relative to the clock edge to avoid race conditions when sampling or driving signals.

III. DEVELOPMENT

The verification environment was built out of three cooperating files, in addition to the RTL itself:

- RTL (`alu.sv`): the DUT. A combinational block (`always_comb`) computes the result based on the 6-bit instruction code, and a sequential block (`always_ff`) registers the result and status flags on the rising edge of the clock, with an asynchronous reset.
- Virtual interface (`vif.sv`): declares all DUT signals from the interface's perspective (signals that are inputs on the DUT are declared as outputs on the interface, and vice versa, since the interface is responsible for driving

the DUT's inputs), together with the clocking block used for synchronized sampling and driving.

- Testbench top (tb.sv): the highest-level module. It generates the clock signal, instantiates the virtual interface, instantiates the test, and instantiates the DUT, connecting the DUT ports exclusively through the interface signals.
- Test (test.sv): defines the stimulus. A reset() task initializes all inputs and asserts reset for several clock cycles; a stimulus task drives randomized operand values (\$urandom_range) while holding the control bits fixed to a selected operation; and a monitor_output() task, launched concurrently through fork/join_none, continuously displays the DUT's inputs, outputs, and flags every clock cycle.

Figure ?? illustrates the connection hierarchy between these blocks.

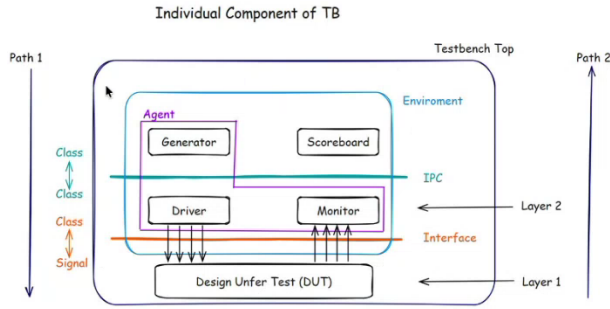


Fig. 1. Connection hierarchy.

This layered structure keeps each component independent: the DUT can be modified or replaced without altering the test, and the test can be extended to new operations without altering the interface or the RTL.

IV. RTL DESIGN

The Arithmetic Logic Unit was implemented at the Register Transfer Level (RTL) using SystemVerilog. The design follows a two-stage structure composed of a combinational datapath and a sequential output stage.

The combinational datapath is responsible for decoding the six control bits and performing the selected arithmetic or logical operation. First, the input operands can be conditionally modified according to the control signals zx_i , nx_i , zy_i , and ny_i . These control bits allow each operand to be zeroed or complemented before the main operation is executed. The signal f_i selects between an arithmetic operation (addition) and a logical operation (bitwise AND), while no_i optionally complements the final result.

The resulting value is produced by an `always_comb` block, ensuring that the output of the datapath responds immediately to changes in the inputs. A simplified description of the dataflow is shown below:

- 1) Apply zeroing operations to the operands if zx_i or zy_i are asserted.
- 2) Apply bitwise negation if nx_i or ny_i are asserted.

- 3) Execute either addition or logical AND according to f_i .
- 4) Optionally invert the final result if no_i is asserted.
- 5) Generate status flags from the resulting value.

The three status flags are generated as follows:

- zr_o : asserted when the result is equal to zero.
- ng_o : asserted when the most significant bit of the result is high, indicating a negative value in two's complement representation.
- pos_o : asserted when the result is neither zero nor negative.

Figure 2 illustrates the RTL architecture of the ALU.

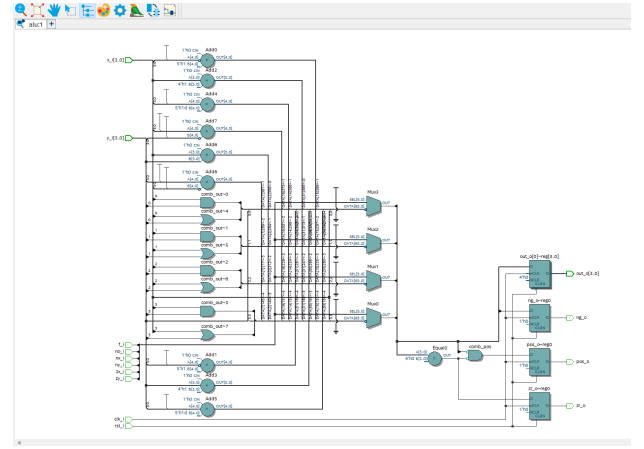


Fig. 2. RTL architecture of the parameterizable ALU.

The RTL design is fully parameterizable through the `DataWidth` parameter, allowing the same source code to support different operand widths without structural modifications. This parameterization facilitates both simulation and FPGA deployment while maintaining the same verification environment.

V. TIMING ANALYSIS

After synthesis, the design was analyzed using the Quartus Prime TimeQuest Timing Analyzer to verify that all timing requirements were satisfied.

Timing analysis evaluates whether signals can propagate through the combinational logic and reach the destination registers within a clock period. The most important metric reported by TimeQuest is the slack value, defined as the difference between the required arrival time and the actual arrival time of a signal.

A positive slack indicates that the timing constraint is satisfied, whereas a negative slack indicates a timing violation. Therefore, the objective is to achieve non-negative slack values for all setup and hold timing checks.

TABLE II
TIMEQUEST TIMING SUMMARY

Parameter	Value
Worst-Case Setup Slack	4.697
Worst-Case Hold Slack	2.583
Maximum Clock Frequency (Fmax)	70.17 MHz

The timing report showed that all timing requirements were successfully met, indicating that the synthesized ALU can operate correctly at the specified clock frequency. The positive slack margin demonstrates that the critical path delay remains below the available clock period, providing reliable operation on the Cyclone IV E FPGA.

Figure 3 presents the TimeQuest Timing Analyzer report generated by Quartus Prime.

	Clock	Setup	Hold	Recovery	Removal	Minimum Pulse Width
1	Worst-case Slack:	4.697	2.583	N/A	N/A	9.440
	vclk	4.697	2.583	N/A	N/A	9.440
2	Design-wide TNS	0.0	0.0	0.0	0.0	0.0
1	vclk	0.000	0.000	N/A	N/A	0.000

Fig. 3. Quartus Prime TimeQuest Timing Analyzer summary.

VI. SIMULATION

The design was simulated using Xilinx Vivado (xvlog/xelab/xsim), driven through a Makefile-based flow. The testbench applied a reset sequence followed by six randomized stimulus vectors with the control bits fixed to the addition operation ($f_i = 1$, remaining control bits at 0).

Table III summarizes representative simulation results, aligning each registered output with the operand values that produced it.

TABLE III
REPRESENTATIVE SIMULATION RESULTS (SUM OPERATION)

X	Y	Out	ZR	POS
0	0	0	1	0
5	9	14	0	1
10	1	11	0	1
14	12	26	0	1
2	4	6	0	1
5	12	17	0	1

All registered outputs matched the expected sum of the operands, and the `zr_o` flag correctly asserted only when the result was zero, while `pos_o` correctly asserted for all non-zero, non-negative results. These results confirm the functional correctness of both the ALU's addition path and its status-flag generation logic.

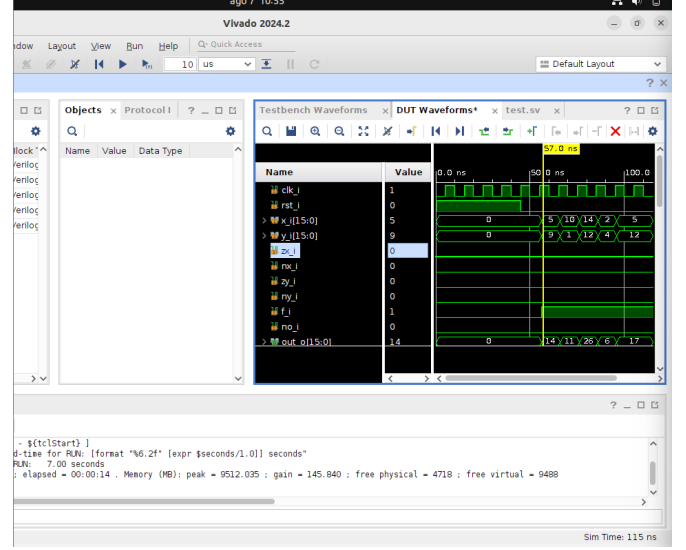


Fig. 4. Simulation console output showing the monitor task display.

VII. IMPLEMENTATION

The verified ALU was targeted to an Altera/Intel DE2-115 development board, based on a Cyclone IV E FPGA, using Quartus Prime for synthesis, fitting, and pin assignment.

Since the board provides only 18 slide switches, the operand width was reduced from the simulated `DataWidth` to a smaller value (4 bits) for the physical demonstration, allowing both operands and the six control bits to be assigned to switches simultaneously without requiring input multiplexing.

Table IV summarizes the pin assignment used for the top-level module.

TABLE IV
DE2-115 PIN ASSIGNMENT

Signal	Board resource	FPGA Pin
clk_i	CLOCK_50	PIN_Y2
rst_i	KEY[0] (inverted)	PIN_M23
x_i	SW[3:0]	PIN_AB28...AD27
y_i	SW[7:4]	PIN_AB27...AB26
Control bits	SW[13:8]	PIN_AC25...AA24
out_o	LEDR[3:0]	PIN_G19...F21
zr_o, ng_o, pos_o	LEDR[6:4]	PIN_E21, E22, E25
carry_o	LEDR[4]	PIN_F18

A relevant implementation detail is that the DE2-115 push-buttons are active-low, while the ALU's `rst_i` input is active-high; this was resolved in the top-level module with a simple inversion (`assign rst_i = ~KEY[0];`).

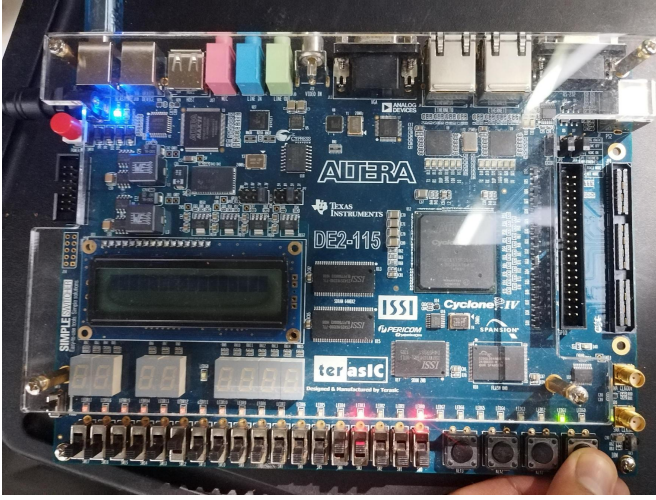


Fig. 5. $X = 1000$, $Y = 0001$, Out = 1001, OP = 000010

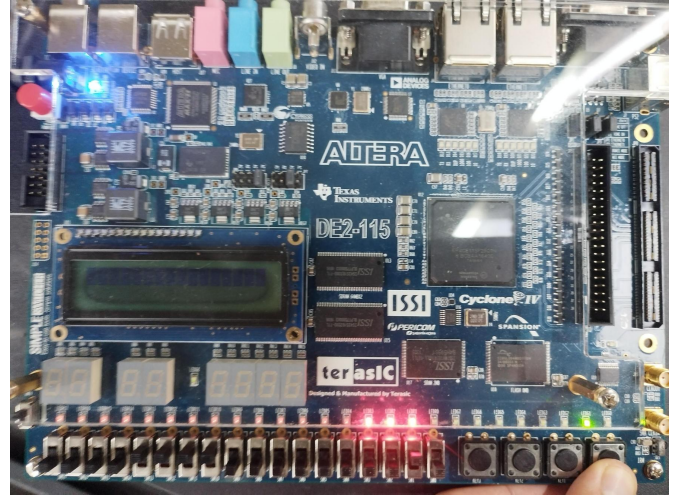


Fig. 8. $X = 0000$, $Y = 0000$, Out = 1110, OP = 001111

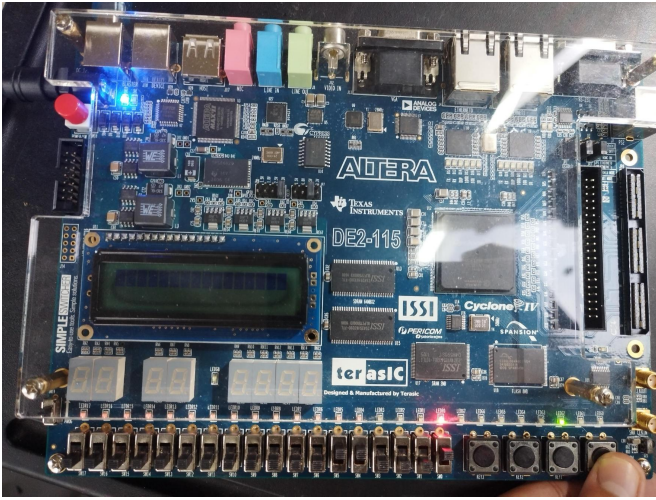


Fig. 6. $X = 0001$, $Y = 0011$, Out = 0001, OP = 000000

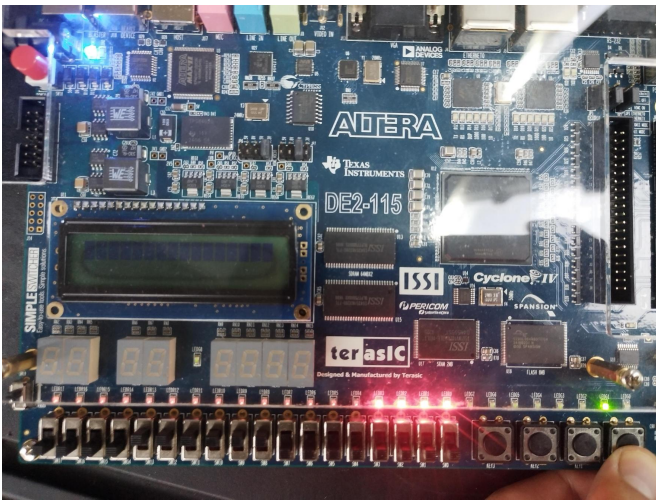


Fig. 7. $X = 0000$, $Y = 0000$, Out = 1111, OP = 111111

VIII. CONCLUSIONS

The verification environment based on a virtual interface successfully decoupled the test logic from the DUT, allowing the same testbench structure originally developed for a simpler adder circuit to be adapted to the ALU with only minor modifications to the interface and test files. Simulation results confirmed that the ALU correctly performs the addition operation and generates accurate status flags.

The subsequent FPGA implementation on the DE2-115 board demonstrated that the design, verified purely in simulation, could be synthesized and mapped to physical board resources (switches, push-buttons, and LEDs) with minimal changes, requiring only a top-level wrapper module and the appropriate pin assignments. This confirms that separating verification concerns (interface, driver-equivalent stimulus, and monitoring) from the DUT itself supports both simulation-based verification and physical implementation without altering the core RTL design.

REFERENCES

- [1] IEEE Standards Association, *IEEE Standard 1800-2023 for SystemVerilog*, 2023.
- [2] Accellera Systems Initiative, *Universal Verification Methodology (UVM) 1.2 User's Guide*, 2015.
- [3] Intel Corporation, *Quartus Prime User Guide: Design Compilation*, 2024.
- [4] Terasic Technologies, *DE2-115 User Manual*, 2013.